



Software Engineering

Tutorial 2 - Software Testing

M. Ianotto

1 Exercise 1

The following algorithm is given for the procedure **insérer** which inserts an integer **x** into the list **L**. The elements of the list **L** are sorted in ascending order. Chaque élément de la liste est constitué d'un champ **info** qui contient la valeur de l'élément et d'un champ **suivant** qui référence l'élément suivant dans la liste.

```
procedure insert(x : integer , L : listElement)
Variables
    current , previous , newElement : listElement
    found    : boolean
Debut
    // initialization of variables
    current <- L
    previous <- null
    found <- false

    // search for the position of the element to insert
    While current != null and found = false do
        If current.info >= x then
            found <- true
        Else
            previous <- current
            current <- current.next
        EndIf
    EndWhile

    // creation of the new element
    allocate(newElement)
    newElement.info <- x
    newElement.next <- current

    // insertion of the new element
    If previous = null then
        L <- newElement
    Else
        previous.next <- newElement
    EndIf
End
```

Question 1 : Draw the control-flow graph of the procédure `insérer`.

Question 2 : Which branches are covered with the following test data :

`x = 4` and `L = [1, 5]`

Question 3 : In which intervals should l'entier `x` à insérer dans la liste `L = [1, 4, 5]` be chosen in order to achieve full branch coverage.

Question 4 : Supposons que, suite à une erreur de programmation, l'instruction `previous.next <- newElement` is replaced by the instruction `precedent.suivant <- courant`. Quelles sont les données de test qui permettent de détecter ce défaut ? Est-ce que le critère de couverture de toutes les branches permet de détecter ce défaut ?

The following C++ code is given for the classes `Element` et `Liste` used to manage the list of integers.

```
#pragma once

#include <memory>

class Element {
private:
    int info;
    std::shared_ptr<Element> suivant;

public:
    Element(int info)
        : info(info), suivant(nullptr) {}

    int getInfo() const {
        return info;
    }

    void setInfo(int info) {
        this->info = info;
    }

    std::shared_ptr<Element> getSuivant() const {
        return suivant;
    }

    void setSuivant(std::shared_ptr<Element> suivant) {
        this->suivant = suivant;
    }
};
```

```
#pragma once

#include <memory>
#include "element.hpp"

class Liste {
private:
    std::shared_ptr<Element> debut;
```

```

public :
    Liste();

    std::shared_ptr<Element> getDebut() const;
    void setDebut(std::shared_ptr<Element> debut);

    void inserer(int x);
};

#include "liste.hpp"

Liste::Liste()
    : debut(nullptr) {}

std::shared_ptr<Element> Liste::getDebut() const {
    return debut;
}

void Liste::setDebut(std::shared_ptr<Element> debut) {
    this->debut = debut;
}

void Liste::inserer(int x) {
    auto nouveau = std::make_shared<Element>(x);

    // Case 1: the list is empty
    // or x must be inserted at the beginning
    if (debut == nullptr || x < debut->getInfo()) {
        nouveau->setSuivant(debut);
        debut = nouveau;
        return;
    }

    // Search for the insertion position
    auto courant = debut;

    while (courant->getSuivant() != nullptr &&
           courant->getSuivant()->getInfo() < x) {
        courant = courant->getSuivant();
    }

    // Insert after current
    nouveau->setSuivant(courant->getSuivant());
    courant->setSuivant(nouveau);
}

```

We want to test the function `inserer` of the class `Liste` using the library `Google Test`. For this purpose, we write the file `ListeTest.cpp` below :

```

#include <gtest/gtest.h>
#include "../src/liste.hpp"

// =====
// Test 1: a newly created list is empty
// =====
TEST(ListeTest, ListeVide) {
    Liste liste;
}

```

```

    EXPECT_EQ(liste.getDebut(), nullptr);
}

// =====
// Test 2: insertion into an empty list
// =====
TEST(ListeTest, InsérerDansListeVide) {
    Liste liste;

    liste.insérer(5);

    ASSERT_NE(liste.getDebut(), nullptr);

    EXPECT_EQ(liste.getDebut()->getInfo(), 5);
    EXPECT_EQ(liste.getDebut()->getSuivant(), nullptr);
}

// =====
// Test 3: insertion in the middle
// =====
TEST(ListeTest, InsérerAuMilieu) {
    Liste liste;

    liste.insérer(2);
    liste.insérer(8);
    liste.insérer(5);

    auto courant = liste.getDebut();

    // TO BE COMPLETED
}

```

Question 5 : Complete **Test 3** to test the insertion of an element in the middle of the list.

2 Exercise 2

The following pseudo-code algorithm is given for the procedure `plusDeC1QueDeC2` qui reçoit en paramètre une chaîne de caractères `s`, and two characters `c1` et `c2`. La procédure affiche un message indiquant si le caractère `c1` appears more often, less often, or as often as character caractère `c2` in the character string `s`. Les indices des caractères dans la chaîne `s` range from 0 to `longueur(s) - 1`

```

procedure moreC1ThanC2(s : string; c1, c2 : character)
Variables
    index      : integer
    c1MinusC2 : integer
Debut
    index <- 0
    c1MinusC2 <- 0
    While index < longueur(s) do
        If s[index] = c1 then
            c1MinusC2 <- c1MinusC2 + 1
        Else
            If s[index] = c2 then
                c1MinusC2 <- c1MinusC2 - 1

```

```

        EndIf
    EndIf
    index <- index + 1
EndWhile
If c1MinusC2 > 0 then
    display("there are more ", c1, " than ", c2)
Else
    If c1MinusC2 < 0 then
        display("there are more ", c2, " than ", c1)
    Else
        display("there are as many ", c1, " as ", c2)
    EndIf
EndIf
Fin

```

Question 1 : Draw the control-flow graph of the procedure `plusDeC1QueDeC2`

Question 2 : Quelles sont les données de test qui permettent d'obtenir la couverture de toutes les branches du control-flow graph.

Question 3 : What result is displayed by the procédure avec the following test data :

DT = {s = "ou", c1 = 'o' and c2 = 'o'}

L'exécution de la procédure avec ces données de test conduit-elle à une anomalie ?

